

```
# mount /dev/cciss/cld0p1 /tmp/1 -t ext2
EXT2-fs warning (device cciss/cld0p1): ext2_fill_super: mounting
ext3 filesystem as ext2

# ls -l /tmp/1

-rw-r--r-- 1 0 0 1724047 Oct 21 07:45 System.map-
2.6.32-4-pve
-rw-r--r-- 1 0 0 106083 Oct 21 07:45 config-2.6.32-4-
pve
drwxr-xr-x 2 0 0 4096 Dec 8 06:54 grub
-rw-r--r-- 1 0 0 10404494 Nov 13 07:17 initrd.img-
2.6.32-4-pve
-rw-r--r-- 1 0 0 10402231 Nov 8 13:42 initrd.img-2.6.32-4-
pve.bak
drwx----- 2 0 0 16384 Nov 8 12:38 lost+found
-rw-r--r-- 1 0 0 124152 Oct 8 2008 memtest86+.bin
-rw-r--r-- 1 0 0 2507616 Oct 21 07:45 vmlinuz-2.6.32-4-
pve
```

The partition that we've mounted is actually the original `/boot` directory. Now we need to transfer the necessary files onto our local Linux machine, and from there, create a mini boot ISO image, specifically designed for our server. How do we do this? We still have an unused virtual USB drive slot. Let's first prepare a loop-back device file on our local system:

```
# dd if=/dev/zero bs=1M count=40 of=blank.iso
# /sbin/mkfs.ext3 -F blank.iso
```

Next, we connect the `blank.iso` image with the `ext3` filesystem to the server USB slot via the Java applet (the *Virtual Floppy/USBKey* option seen in Figure 3). After this, tailing `dmesg` on the server console shows up something like the following code:

```
usb 5-2.1: new full speed USB device using uhci_hcd and address
10
usb 5-2.1: New USB device found, idVendor=03f0, idProduct=1427
usb 5-2.1: New USB device strings: Mfr=1, Product=2,
SerialNumber=0
usb 5-2.1: Product: Virtual Floppy
usb 5-2.1: Manufacturer: HP
usb 5-2.1: configuration #1 chosen from 1 choice
scsi8 : SCSI emulation for USB Mass Storage devices

# scsi 8:0:0:0: Direct-Access HP Virtual RayDrive 0.01 PQ:
0 ANSI: 2
sd 8:0:0:0: Attached scsi generic sg2 type 0
sd 8:0:0:0: [sda] 81920 512-byte logical blocks: (41.9 MB/40.0
MiB)
sd 8:0:0:0: [sda] Write Protect is off
sd 8:0:0:0: [sda] Assuming drive cache: write through
```

```
sd 8:0:0:0: [sda] Assuming drive cache: write through
sda: unknown partition table
sd 8:0:0:0: [sda] Assuming drive cache: write through
sd 8:0:0:0: [sda] Attached SCSI removable disk
```

Now we mount the newly detected (virtual) USB drive:

```
# mount /dev/sda /tmp/2 -t ext3
```

We copy the kernel, the `initramfs` file with integrated drivers, and settings from the GRUB loader into the mounted image, `sync`, to ensure that buffers are flushed (data is written across the network onto our disk) and we then unmount our virtual USB drive:

```
# cp /tmp/1/initrd* /tmp/2
# cp /tmp/1/vmlinuz* /tmp/2
# cp /tmp/1/grub/menu* /tmp/2
# sync && umount /tmp/2
```

On the local machine, we disconnect the ISOs from the virtual USB and virtual CD using the *Virtual Media* tab of the Java applet. Now, we mount this image file locally, and with the help of the Syslinux utility [5], create a bootable ISO image. First, we create a directory structure as follows:

```
./isolinux/isolinux.bin
./isolinux/isolinux.cfg
./vmlinuz-2.6.32-4-pve
./initrd.img-2.6.32-4-pve
./grub/menu.lst
```

The generated directory tree for the mini ISO image should conform to Syslinux rules. Next, we launch the `mkisofs` command with these options:

```
# mkisofs -o mini_iso.iso -b isolinux/isolinux.bin -c isolinux/
boot.cat \
-no-emul-boot -boot-load-size 4 -boot-info-table CD_root
```

Booting with the new mini-boot ISO

We connect the new `mini_iso.iso` as a virtual CD drive on the server, and in the iLO page's *Administration* tab, click the *Reset* button. We switch to the virtual console and watch the system boot messages. As is visible in Figure 9, the system is intact, and boots as it should. Cool! Now we need to point the boot loader to a new location—to be precise, to the beginning of the `/dev/cciss/cld0` device, where it was logically expected, but it wasn't present. But first, let's verify what we can see at the beginning of the second disk:

```
# hexdump -C /dev/cciss/cld0 | less
00000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```